1 Key code components

Basic description of the components of the code (probably each will form its own namespace).

I. **mesh**: Will form a namespace which will accommodate classes for the mesh structure. Keep in mind that there will also be a structure that explicitly generates a regular mesh $M \times N$ with a given genome.

II. **basis functions**: Block dedicated to the class that has the edge-detection algorithm, divergence-conforming basis-function class that manages the indexing of the basis functions. Next there will be concrete variants of the divergence-conforming basis functions called rooftop basis functions. Two variants will exist: one that specifically works with the genome of the mesh structure, and a second that ignores the genome and rapidly generates the appropriate structures.

III. **core**: A more general name would be "MoM strategy," because it will not only include the impedance-matrix assembly process—there are solvers (iterative, e.g. CGFFT) that do not require the impedance matrix. The top-level class will decide which strategy to use based on user input and the desired output (impedance matrix, current vector, etc.). The input will be the appropriate excitation.

   i. **memory manager**: Decides the maximum sizes of blocks, vectors, and matrices for subsequent planners.

   ii. **impedance-matrix planner**: Contains several variants of the planning process. Each class produces a structure that accommodates any information needed for the next stage of assembly—basically the indices of the matrix that need to be computed in each iteration and how.

   iii. **impedance-matrix assembler**: Takes the precomputed chunks from the planner and assembles the desired components of the impedance matrix. Consists of a core algorithm specialized in computing the actual impedances.

   iv. **MATLAB's built-in solver**: Produces the final current vector based on the specified excitation.

   v. **Green's-matrix planner**: Similar to the impedance-matrix planner; it determines which indices to generate at once.

   vi. **Green's-matrix assembler**: Assembles the matrix representation of the Green's operator in a format acceptable for subsequent calculations.

   vii. **conjugate-gradient solver**: Solves for the currents given an excitation using the conjugate-gradient method. The preconditioner will be implemented in a separate class.

IV. **antenna operators**: Likely contains the excitation vector (the input to the MoM), represented as the planar-wave operator matrix. This namespace will also include RCS, far-field operators, input impedance, etc.

V. **visualization tools?**

2 Code structure

Here we will expand the whole code structure down to a single function, each will has a brief description what it does. This will not only clarify the code oritentation, it will also motivate each component existence and usage, which should be homogenous with SRP. It will also extinct the duplicity and wrong context of the component. Each (sub)subsection will represent namespace. Then each of the top-level enumerates will represent class names and the subsequent indented enumerated will list all methods of the class.

3 `rooftopMoM`

This is the top-level namespace of the project. The class `RooftopMoMStrategy` will be the user interface with the whole package. The settings of the compute strategy is placed in `RooftopMoMStrategySettings`. But the user will be also be able to create a custom wrappers/decorators of the internals so no strict encapsulation will not be restricted.

I. `Strategy`: The top class — decides what strategy of the computation to choose — it will disponate with several constructors, including the empty one for fully custom approach.

   i. `constructor`: There will be three levels of constructors: first the empty one, so the user must use the load methods to use the MoM properly, the second one will accepts only the setting structure and last one will accepts the settings

   ii. `loadMesh`: Load the mesh with the structure of `mesh/Mesh` or `mesh/MeshCluster`, basically any structure with vertices list and apropriate face list with the uniform rectangular structure will be accepted.

   iii. `loadUniformRect`: Accepts the `UniformRect` abstraction through which the `Mesh` or if the input is cell of `UniformRect` it will produce the `MeshCluster`.

   iv. `loadUniformRectPrimitive`: Loads the primitive and assumes that the origin of the mesh is at $(0, 0, 0)$ and all Euler angles are zero.

   v. `loadRooftopBasisFunctions`: Loads the `basisFunctions/RooftopBasisFunctions`, the mesh is not required because it is accesible through the `mesh` handle.

   vi. `loadFastRooftopBasisFunctions`

   vii. `loadSettings`: Insert the instance of `RooftopMoMStrategySettings` manually.

   viii. `generate`: Initiates the private function `checkRequirements` if it suceddes it will create necessary strucures to compute the desired objects and returns general instance of `RooftopMoMStrategyResult` which hold necessary informations.

II. `StrategySettings`: Here will be placed all the settings required for deterministic behaviour of the MoM. The idea is that based on these tags we decide what needs to be computed and what is not: *impedanceMatrixElectricPart*, *impedanceMatrixMagneticPart*, *impedanceMatrix*, *currentsInverse*, *currentsCGFFT*, *currentsPCGFFT*, *currentsGMRES*. There will be more settings if we desire.

   i. `constructor`: This will be parameteric constructor. Three level constructor aproach.

   ii. `setTarget`: Parameteric, but for the first time i would make it such that one target is required.

   iii. `target`: Returns the target name.

   iv. `setMemoryMiB`: Give the MoM maximal amout of the memory for the computation in MiB. Other units will be considered.

   v. `memoryMiB`: Returns the given memory in MiB.

   vi. `setMemoryGiB`

   vii. `memoryGiB`

III. `StrategyResult`: This class will has every possible output of the MoM stored in it, potentially the times of each of the crucial operation as well, the data will be structuralized as a dictionary. For each of the methods will returns a structure that will has commonly two main fields: *data*, `timeOfExecution`, and potentially others ie. `iterationCount`, `relativeResiduals`, ... If the desired output is not accessible then it was not computed and it marked as *nan*.

   i. `impedanceMatrixElectricPart`

   ii. `impedanceMatrixMagneticPart`

   iii. `impedanceMatrix`

   iv. `currrents`

3.1 `mesh`

All classes asociated with the mesh generation will be placed here. Up on these structures will be placed the classes from the basis function, everything about the geometry of the problem will live here. Therefore we will introduce a structure(s) that will form a bridge to the basis function class.

I. `UniformRectPrimitive`: This is a entry structure that will hold the dimensions of the elementary cell and how many are those in each direction. The basic unit will be 1 m for now, in future there will be also a posibility to use relative unit system.

   i. `countX`

   ii. `countY`

   iii. `count`: Returns the total number of patches in the mesh.

   iv. `sizeX`

   v. `sizeY`

   vi. `meshSize`: Returns the geometrical size of the mesh in a pair.

II. `UniformRect`: Takes the `UniformRectPrimitive` and places the mesh in 3D according to the bottom-left corner (in the future add more variety of rect origins), as well three euler angles which will orient the rectangular plate. The output of this structure is the universal Mesh, but with optimal basis function ordering (important for best acceleration techniques performance).

   i. `constructor`: There will be 2 types of constructor default — the user provides only the `UniformRectPrimitive` — and the unit determinant transformations are not used. Second will require all transformation parameters, origin and three Euler angles.

   ii. `generateMesh`: Outputs the mesh of the primitive with optimal ordering of the faces.

III. `Mesh`: The central structure of the `mesh` namespace will hold the mesh structure in two arrays, so by a list of vertices and list of faces (each face will be connected to n vertices). There is a small problem, if the interaction symmetry algorithm has to be optimally perfoment we will prefer some determined scheme for the face ordering because it will influence the basis function ordering.

   i. `vertices`

   ii. `faces`

   iii. `setVertices`

   iv. `setFaces`

   v. `count`: Return 1, here is due to the exchange with `MeshCluster` which will has `count` implemented.

IV. `MeshCluster`: Multiple `Mesh` structures can be accommodate here. The reindexing of the vertices and faces is automatical. Each mesh is represented as a element of a cell. The point is that the `MeshCluster` is interchangible with the `Mesh` class.

   i. `get`: Returns a mesh instance for a given index.

   ii. `insert`

   iii. `remove`

   iv. `vertices`

   v. `faces`

   vi. `count`: Returns the total number of `Mesh` instances in the cell. Useful for iterating through the cluster.

V. `utils`: Namespace that bundles functions that operates with the `Mesh` and `MeshCluster` objects

i. `applyGenomeToMesh`: Function that takes the `Mesh` instance and applies the genome (binary array of face count), it returns new `Mesh` instance with adjusted faces, but the vertices array is the same for

ii. `generateInnerEdges`: Function that will return a 2 x N array of patch indices sorted (for better acceleration performance) from given mesh object.

iii. `generateGridSizes`: Function that for `Mesh` and `MeshCluster` generates apropriately sized grids for each of the mesh elements. Returns it as a cell of pairs. The prerequisity is that the meshes are plannar structures.

3.2 `basisFunctions`

As described in the section of key components, here will be placed all classes that generates from the `Mesh` or `MeshCluster` the two key structures: indices list and points. The indices list will be generated for divergence conforming basis functions directly from the mesh based on the edge detection algorithm. Each inner edge of the mesh is associated with one divergence conforming basis function.

I. `UniformRectDivergenceConforming`: This is a special case of the divergence comforming basis functions based on uniformly discretized mesh into rectangular patches, other type very popular are RWG basis functions.

i. `constructor`: Takes the `Mesh` or `MeshCluster` instance and saves a reference.

ii. `generateIndices`: calls the function `generateInnerEdges` from `mesh/utils`

iii. `count`: Returns the total number of basis funcitons.

iv. `countDir`: Returns the number of basis funcitons in given direction.

v. `faces`: Returns sorted faces first the X direction then Y direction.

vi. `facesCell`: For each mesh in cluster new cell.

vii. `facesCellDir`: Returns only specific section of the basis functions based on the direction.

viii. `generateIDMask`: Generates a mask ID mask, each of the basis functions (column of the structure), is given a number based on its mesh location. Used later for mesh size adjustment.

ix. `mesh`: Returns a handle to the internal mesh (read-only ideally but in MATLAB it is not possible I am affraid).

II. `RooftopBasisFunctions`: This class derives from the `UniformRectDivergenceComforming` and generates the point matrix for each rooftop based on the indices of the basis functions, one by one. The class also contains the quadrature points for each of the rooftops.

i. `constructor`: Requires the instance of `GaussLegendre2RectQuadrature` and `Mesh` or `MeshCluster` for the base class.

ii. `generate`: Triggers the `generateIndices` function from the base class. Then it generates the points of the given quadrature.

iii. `generateQuadraturePoints`: Computes the shifted quadrature points for each of the basis functions.

iv. `generateRooftopPoints`: Computes the rooftop points for all given basis function indices.

v. `generateRooftopDivPoints`: Computes the divergence of the rooftops.

vi. `rooftopSizes`: For each of the rooftops gives domain rectangle dimensions, each size is given an index, which will be used for construction of the jacobi mask.

vii. `rooftopSizesUniquePerms`: Returns the vector of unique rooftop sizes, permutations of the sizes included (needed for the singularity extraction).

viii. `rooftopSizesUniqueCombs`: Returns the vector of unique rooftop sizes, unordered, used for jacobi mask.

III. `FastRooftopBasisFunctions`: This one does not derive from the `UniformRectDivergenceComforming`, it is used for the superfast CG-FFT algorithms.

   i. `constructor`: The input can be the `Mesh`, if we are able to deduce the gridsize of the mesh based on the `Mesh` structure. If not then the instance of `UniformRectPrimitive` will be used.

   ii. `count`: Returns the number of basis functions for given grid.

   iii. `generateRooftopPointsDir`: Returns the rooftop points for rooftops in certain direction.

   iv. `generateQuadraturePointsDir`: Returns the quadrature points for rooftops in certain direction.

   v. `generateQuadraturePoints`: Returns all quadrature points, used for electric part of the impedance matrix.

   vi. `meshPrimitive`: Returns (should be read-only) handle to the `UniformRectPrimitive`.

IV. `GaussLegendre2RectQuadrature`: Class that will be used for generating the quadrature weights and quadrature points for a pair of rectangular domains. Each quadrature order will has different class instance.

   i. `order`: Gives the quadrature order.

   ii. `generate`: Generates the weights and points for two unit squares.

   iii. `weigths4D`: Obtain the weights for 4D integration.

   iv. `weigths2D`: Obtain the weights for 2D integration.

   v. `points`: Returns pair of points, each one for one rectangle.

3.3 `core`

This namespace is dedicated to the computation of the impedance matrix, currents — basically the core components of the whole code. Very important aspect of the core is memory management. For every method of moments it is crucial to be able to manage the memory usage of the matrix, operator assembly.

I. `MemoryManager`: Based on what the MoMStrategy chooses to compute, the memory manager will comute required block size (maximal count of doubles for a given quadrature). It will also consider the memory needed for the accompanioning matrices.

II. `Constants`: This class will host all necessary constants for the MoM, so speed of light, vacuum permitivity, vacuum permeability etc. It will has multiple constructors based on the desired input (most common is wavenumber, but ka will be also desired).

3.3.1 `impedanceMatrix`

All about the computation of the impedance matrix, its actual assembly will be placed in this namespace. The `ComputeContext` will serve as a holder of necessary classes for the impedance matrix construction process. The base class `ComputeStrategy` has the common function for all derived classes which will compute the impedance matrix.

I. `singularIntegrals`: This namespace will accommodate the analytic formulas for the singular integrals.

   i. `type1`

   ii. `type2`

   iii. `type3`

II. `InteractionSymmetriesAccelerator`: This class will implement the interaction symmetries approach of accelerating the impedance matrix computation. It will basically serves as a decorator of the Assembler stack.

i. `generate`: Returns the part of the impedance matrix based on the instance input (`FullAssembler`, `MagneticPartAssembler`, ...) and the given `AssemblerContext`. It will basically wrap around the `generateImpl` of the objects injecting the custom computed interaction and consequently filling the matrix using the symmetries.

ii. `generateContext`: Returns the mask, interactions and chunks. It is a private function.

III. `InteractionSymmetriesAcceleratorImpl`: Class that helps to separate the internals and utility functions of the interaction symmetries aceelerator. This class implements the main logic for the interaction symmetries approach. It is created at the same time as the `InteractionSymmetriesAccelerator`

i. `generate`: Returns the computed impedance matrix masks and the arrays of interactions that yields unique interaction impedance value.

ii. `generateSingle`: Returns a single mask and interaction array.

iii. `apply`: Applies the mask and returns the fully occupied matrix.

iv. `generateToeplitz`

v. `generateBothDir`

vi. `generateSameDir`

vii. `generateCrossDir`

IV. `Assembler`: Base class for impedance matrix assembly classes. It will also host necessary functions for the common matrix assembly process - singularity mask generator etc. It will also has common constants saved in it (polarity mask, ...).

i. `generate`: An abstract method that must be implemented by all derived classes.

ii. `generateSingularityMask`

iii. `generateIDMask`

iv. `generateJacobianMask`

V. `AssemblerChunkPlanner`: Plans the chunk scheme for the Assembler. It can generate the chunk scheme for all chunks in the matrix, or based on the given interaction scheme, used for selective matrix rendering. It will be initialized with the `Assembler`, therefore its methods accesible to all derived classes.

i. `constructor`: Needs only the double count (block size) for the subsequent calculations.

ii. `generate`: Based on the given double count in one block it will produce the chunk scheme for the whole impedance matrix.

iii. `generateSelected`: It will return the chunk scheme based on the given interaction matrix.

iv. `chunksToSlices`: Converts the chunks into slices for the distance matrix computation.

VI. `AssemblerContext`: All necessary classes for the actual computation will be placed here.

i. `quadrature`: Returns the internal quadrature reference.

ii. `basisFunctions`: Returns handle to the basisFunctions.

iii. `constants`

VII. `MagneticPartAssembler`: Assembles only the magnetic part of the impedance matrix.

i. `constructor`: The input is the memory requirements.

ii. `generate`: Implements the base class abstraction. Input will be only the `AssemblerContext`, so in the future we can call on one instance of the assembler multiple antenna wavenumbers or electrical sizes.

iii. `generateImpl`: Implementation of the assembly, but the additional parameter of the method it the interaction matrix for selected parts of the impedance matrix calculations.

iv. `generateChunk`: Private method implementing the chunk computation.

v. `prepareSingularityContext`: Computes the necessary arrays for singularity evaluation.

vi. `computeSingularities`: Computes the values of the extracted singular integrals using the functions in the `singularIntegrals` namespace.

VIII. `ElectricPartAssembler`: Assembles only the electric part of the impedance matrix.

i. `constructor`

ii. `generate`

iii. `generateImpl`

iv. `generateChunk`

v. `computeSingularities`

IX. `FullAssembler`: Assembles full impedance matrix.

i. `constructor`

ii. `generate`

iii. `generateImpl`

X. `utils`: Namespace that governs any utility functions for the `impedanceMatrix` namespace.

i. `generateIntegralKernel`: Produces the integral kernel matrix for the impedance matrix assembly.

ii. `generateDistanceMatrix`: Used for the scalar Green's function computation.

3.3.2 `greensMatrix`

This namespace contains essentially the same algorithms (potentially can be condensed into one namespace with a different inputs).

I. `Assembler`: Basically the same role as in the i

II. `AssemblerChunkPlanner`: Plans the chunk scheme for the whole code.

III. `ElectricPartAssembler`:

IV. `MagneticPartAssembler`:

V. `FullAssembler`

VI. `utils`: Namespace for helper functions.

i. `toeplitzToCirculant`:

3.3.3 `solvers`

Here will be placed all the solvers, ie. the algorithms that will compute the current vector.

I. `Inversion`

II. `ConjugateGradient`

III. `ConjugateGradientSpectralPreconditioner`

IV. `GeneralizedMinimalResidual`

3.4 operators

This namespace will contain some obligatory operators for proper `core` function, like the generation of the excitation vector. Then there will be operators that will utilize the outputs of the `core` like the impedance matrix for input impedance, or the current vector for `FarField` or `RCS` and others.